Kutty Banerjee

kutty.banerjee@gmail.com (925)-209-7204 (US Citizen)

Summary

- Experienced in Kernel Drivers, Firmware, Perf and Power Analysis, System Wide Optimizations, Leading Cross-Team efforts across SW-HW boundaries.
- Was the Windows Kernel Driver Lead for NVIDIA's Fermi Generation of GPUs.

Experience

Apple: iOS GPU Engineer [Aug 2013 – Present]

- Added in Firmware, a Multi Linear Regression based Model for predicting Runtime GPU Power Consumption.
- Augmented Firmware with a Credit based Policy for Fairness in GPU Work Scheduling.
- Added GPU Compiler and Driver support to implement Efficient Vertex Clipping for User Planes (OpenGL)
- Added Data flow logic in Driver to Elide Cache Flushes between OpenCL kernels missing RAW/WAW dependencies

Intel: GPU Platform Architect [May 2013 – Aug 2013]

- Drive Low Power design of Compute features on next-gen Intel GPUs. Use models to predict Perf and Power.

NVIDIA: Software Architect [May 2005 – May 2013]

Project Denver (NVIDIA's ARM64 CPU)

- Lead Android Stack optimizations for Denver CPU targeting Linux Kernel and Browser JIT improvements.
- Did CPU Perf Analysis of Graphics, Web and JVM workloads and coordinated with various HW and SW teams.
- Helped triage CPU Performance problems due to various Cache and Instruction Scheduling stalls.
- Implemented a **Memory Profiler** and augmented CPU/GPU Functional Simulators to use it with a ~15-20% runtime overhead.
  - Captured Physical/Virtual Pages accessed from various threads, cores along with Page attributes.
  - Provided offline tools for analysis and graph plotting and studying Cache Aliasing problems.
- Denver uses a **Firmware layer** that converts ARM binaries to VLIW on the fly using Profiling and Background JIT Compilation.
  - Maintained the **Profiler, Memory Management** and **Thread Scheduling layers in the Firmware**.
  - Improved Thread Scheduling fairness across Background Compilation and Garbage Collection threads.
  - Improved Power Management support for transitions between various Power States.
  - Added Compiler attributes to prevent Interrupt and non-interrupt code from calling each other.
  - Added Compiler attributes to enforce time **Lock Ordering Rules at Compile Time** instead of Deadlocks at Customer sites.
  - Improved the state-dump feature for offline dump analysis of the Runtime.
  - Added **Lock Stat**, a feature that dumps out status of various locks in the system along with their Usage Statistics.
  - Added support for **Deadlock analysis** on Live running systems and on Crash Dumps.
- Added **Large Page Support in Linux kernel** using a BSD-like Lazy Page Coalescing approach for ARMv7.
- Added an extra **Thread Local Memory Type** in the Tegra Linux Kernel
  - Added kernel support to elide TLB invalidates broadcasts and cache snoop for such memory regions.
  - Useful for Browser workloads on the Android stack. Boosted Google Chrome and Mozilla by ~20% on the V8 benchmark.
- Added Denver specific Cache flush/invalidate changes for Instruction and Data cache in the Tegra Linux Kernel branch
- Worked with various Driver teams to enforce code correctness for ARM Weak Consistency Rules and AXI Bus ordering rules.

Project Orion (CUDA) – Cache Coherent CPU/GPU on a single SoC

- Lead Architect tasked with driving UVM (Unified Virtual Memory) implementation across CPU and GPU on a Custom NVIDIA SoC with an internal CPU GPU and Cache-Coherent Interconnect for the CUDA HPC Market.
- My role was to drive adding features in GPU, SoC and the Driver stack to make this happen.
- This included driving support for
  - GPU Page Fault handling, GPU TLB redesign, Compression and DMA logic for Page Migrations, Fault Interrupt coalescing
  - Routing GPU interrupts from CPU to an onboard Microcontroller for Faster Interrupt Latency.
- Designed ARM64 Linux kernel changes to make this possible. Prototype changes included tapping into the Page Table setup code to do broadcast setup/invalidate for CPU and GPU Page Tables (very similar to **Shadow Page Tables**)
- Added a Radix Tree (Inverted Page Table) in SW to maintain Physical Page and GPU Virtual Page Address correlations for quicker Page Invalidation, Page Unmapping and Page Sharing.
- Implemented GPU Functional and Perf Simulator changes to test Performance impact of False-sharing, Page Thrashing, GPU TLB invalidate performance.
- Project was later cancelled but the above work moved to a different project

GPU Test Infrastructure Lead

- Lead a Cross-Team effort to unify verification efforts of various Hardware and Software Kernel Driver teams on the MDIAD-MODS Platform.
- Added "Probe Based Verification" to Policy-Manager, which is the layer responsible for introducing faults/events and boundary conditions that allow testing hard-to-hit conditions deterministically from an external script.
  - Probes are hooks in Kernel Drivers, which can be programmed at runtime using a script to wait for asynchronous events to occur. Allows an external script to force certain hard-race conditions to occur and ensure Driver works correctly.
  - Power Management Transition Module was the biggest consumer of this Verification Feature.
- Coordinated with Hardware and Software teams working on PCI-E, Display, Cache/TLB, Clocks, Core-Graphics and Video to beef up verification using the above Framework.
- Merged SW use cases with RTL HW unit Test Bench to get more directed coverage

Fermi Generation GPU (DX11 Generation) Windows Kernel Driver Lead

- As the Kernel Driver lead, **lead a team of 25 engineers in the Resource Manager Team** and oversaw development across Simulation, RTL, Emulation, Bringup and Product Release.
- Lead the Fermi GPU Bringup Efforts on x86.
- Role included Task Assignment, Initial Triage, Interaction with Hardware and other Software teams, mentoring new Engineers and Code Review. Responsible for overall progress of the Kernel team for Fermi.
- Was the first engineer to start work on the Kernel/Firmware for Fermi. Wrote and helped large parts of the kernel code including Thread Scheduling, Interrupt Handling, Memory Management, Page Table Setup, Context Switch Support using Firmware, Cache mgmt, **TLB invalidate batching**.
- Added a Performance critical path to elide TLB invalidates completely by ensuring Heap allocator did not reuse Virtual Addresses using an Arena Allocator.

Fermi Command Buffer Scheduling Lead

- Lead the Fermi Command Buffer Scheduling unit in the Driver. Fermi added Hardware support for DX11 Per Node Command Buffers. Replaced the old Time Slice Based Scheduler with Per Node Queues.
- Did plenty of code refactoring and wrote lots of tests. Scheduling module had around 90% branch Code Coverage!!

Fermi Driver Power Management

- Added Driver support for Graphics and Video engine Power Gating, Clock Gating.
- Improved P-state switching support and memory clock switching support in Microcontroller.
- Guided new team members on porting Power Management support entirely to the Microcontroller.
- Wrote tests for Power State transitions and boundary condition testing.

Resource Manager Graphics Driver

- Worked on various parts of the kernel driver, fixed bugs, did triaging.
- Parallelized the Windows Kernel Driver Makefile and added support for distributed Builds.
- Implemented a **Non Stalling Lock Free Interrupt Tree** for routing DXG kernel notifiers without delay

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug – 2005 May]

- Thesis: Remote Execution of 3D Graphics for Mobile Systems.
  - Implemented a Memory and Power Model to Predict Consumption for Graphical Workloads running on Mobile Devices.
  - Recipient of WPI Institute Fellowship Award for outstanding research work. Publications in Journals and Conferences

BE Computer Science, University of Mumbai [1999 July – 2003 May]

Publications http://www.pseudoparallel.com/publications

Languages C / C++ / Java / Objective-C / SystemC / Rust (learning)